

How To Generate Consistent Game Assets with AI

This document is based on the YouTube video: [How To Generate Consistent Game Assets with AI - GPT 5.4, Sora 2, Image 1.5, Codex CLI](#).

Project Overview

This video demonstrates how to generate custom, animated game assets without leaving the terminal, using AI tools to solve character consistency.

The Stack

- **Art Generation:** GPT 5.4 (GPT Image 1.5) for high-fidelity base characters and "Anchor Frames."
- **Animation:** Sora 2 for complex physics-based movement (walk cycles, death animations).
- **Engine:** Phaser JS for the game logic and asset integration.

Key Concepts Learned

- **The Anchor Frame Strategy:** Generating a base character that the AI references for perfect consistency.
- **Sora 2 for Spritesheets:** Using video generation models to handle the heavy lifting of character movement.
- **The "No-App" Workflow:** Using the Codex CLI to prompt art directly into game files without opening Photoshop or a browser.
- **Normalization:** Techniques to clean up AI-generated spritesheets and make them game-ready in seconds.

Video Timestamps

Timestamp	Topic
00:00	Intro
01:36	VibeGameDev.com
02:35	Project Overview
04:02	Step 1: Start with your whole game concept
06:05	Step 2: Generate a single character anchor frame
10:41	Step 3: Idle animation using pixel shifts
12:13	Step 4: Generate animation spritesheets
13:26	Step 5: Normalize animation spritesheets
16:24	Step 6: Limitations for walk cycles

Timestamp	Topic
17:38	Step 7: Using video generation models (Sora 2) for animations
21:14	Step 8: Project Setup
23:11	Step 9: The prompt and plan
25:09	Step 10: AskUserQuestion tool
30:13	Step 11: Saving the prompts and plan
32:00	Step 12: Version control with LazyGit

Transcript:

Hey everyone. In my last video I showed you how to vibe code 3D games using three JS, but for today we are going 2D I vibe coded these 2D games using agent skills, just like last time, an RTS A 2D RPG prototype, and this pixel platformer. And once again, these started to blow up all over Twitter and everyone wanted to know how I did it. And so in today's video, I'm going to show you how to vibe code 2D games, and we'll be rebuilding this platformer that you see here I'll be walking you through everything from start to finish and this workflow that I'll be showing you will work with Cloud Code Cursor or Codex, whatever your favorite vibe coding tool is.

and the key unlock as before is agent skills. I built a phaser JS skill, which is a 2D game engine, and it will handle everything for you from the spritesheets to the physics, to the animation, and basically all the stuff that would take you hours if you were doing it manually.

And as a bonus, I will be showing you a second skill, which is a playwright game testing skill. This gives the AI eyes to be able to see what is being built. It can automatically open up the browser. Press keys and effectively play the game for you and test to see if the functionality works. So if you've ever wanted to learn how to vibe code 2D games, but could never get started, this is the video for you. If you want the exact skills, prompts, and source code from today's

This video's resources and more...

tutorial, I've made them completely free as part of my insiders club. I also send out weekly emails with tips, workflows, and early access and previews to all the stuff that's coming up. And if you want to go deeper, check out builder pack.ai. In fact, I just relaunched it to version two.

It includes my entire VI coding workflow, configurations, agents skills. Commands and basically all of the resources and source code from all of my videos that you see on YouTube there's also a private Discord community where I share some exclusive updates and where other vibe coding builders hang out in order to exchange ideas and to share insights and soon to come will be two additional resources.

One of them will be specifically on AI game development. So topics like what you're seeing in today's video. and a second one will be about how to vibe code iOS apps. so these will be landing soon and the price will be going up. So if you lock in today, you will automatically get hold of these two resources at today's price. the link is in the description below. And without further ado, let's get started with today's video.

Step 1: Agent Skills & Project Setup

All right, we are ready to get started so you can see that all I have here is a blank Visual Studio Code project.

The only two things that we have over here are this .claude and .codex folders, and if we just jump into them, you will see that the only things that are present are the skills that we'll be using.

So the first skill, which is the phaser, game dev skill, is the key one, which will enable us to be able to build these phaser js based 2D games.

the whole point about this skill is that it's been something that I've created and then I've iterated upon and I've built more and more projects.

And just going back to this question about how I build skills, I actually have a skill creator skill, and this is what I use all the time

whenever I'm building a new skill. So whether it's for three js or for phaser js, and you can get hold of this skill creator skill from Philanthropics very own. GitHub repository, but it's worthwhile noting that I don't actually use that one.

I've actually built my own version called Skill Creator Plus, and this one is a more advanced version that I've trained on several different

skills that I've seen, and I find that it's more effective and it's been very useful for me to build skills such as this phaser skill or this playwright

testing skill that you see over here. If you wanna get a hold of that, It's at builder pack.ai. And you can check it out in the link in the description below,

but you don't need any of that for this tutorial because, all of these skills I've made available for free.

So you'll be able to follow along with this tutorial completely, without having to, get hold of anything else.

Okay, so you'll also, you might be asking, why do I have two folders? one of it is the Skill Folder for Codex, and the other one is for clo.

Now, Codex actually uses an open standard for skills, and this one is. Technically cross compatible with everything else, open code or, or any

other, ID that you have out there. Whereas the one has a slightly different format. It is worthwhile noting though that if you are using something like

Cursor, to over here, you can actually make use of these skills, as well. so if you open up Cursor and you go down to this rule Skills and

agents, you will see that Cursor is able to pull in the skills from clo. So This entire tutorial will work completely fine, whether using Codex,

whether using CLO code, whether using Cursor, and it's entirely your choice. the version that you see over here, which has this parallax scrolling

backgrounds, the player can attack and you can see all of the animations are working correctly as well as the ting and all the props, all work out fine.

if you want to. This version was built using GPT 5.2 and GPT 5.2 Codex. I had really good success with this.

And If you've been following Twitter, I think a lot of people are starting to come to realize how powerful 5.2 is, especially when you run it with high reasoning.

but what I'm gonna do today is I'm gonna try to make use of Claude Code and Opus 4.5 to build something similar.

And let's just see how well it does. I'm going to jump back into Visual Studio Code The other skill, that we'll be making

use of, is this playwright testing skill. Now this one is a skill CLO code to be able to automatically test your game for you. So if there's a bug, it will be able to identify the bug for you and then try to fix it. And it's a really cool process and I think this is.

one of the key unlocks of being able to make use of that. In order to make use of this skill, you will need to install the playwright MCP server.

it's completely free and it, it's really simple to install. You just have to add this to your, j file. so I think we're ready to begin so that we can see if we can get by the end of this tutorial to something as close to this. so what I'm gonna do is I'm gonna show you the exact process that I took to get to the stage.

And with that, you can take it and continue iterating upon it and adding new functionality. But my whole goal is to just demonstrate to you how easy it is to vibe code 2D games like this using agent skills. so without further ado, let's get started.

Step 2: Art Assets

All right, so we are back in Visual Studio over here. What I'm gonna do is I'm just gonna collapse everything on the left and close this off.

So the first thing I want you to do is that we need to prepare our folder structure for the art assets. Okay?

So we will create a new folder, and I'll call this public. And I like to create another folder within that called assets.

This folder structure over here just enables us to put all of our assets into one location and so that when we deploy this onto the web, it will be able to be ready for, people to play. If you don't put this, properly, you will have loading errors because it

wouldn't be able to locate the, assets when it's loading it from a web server. So anyway, if you did not understand any of that, don't worry.

Just know that we are gonna put everything, to do with our art assets into this folder over here And speaking of art assets, let's take a look at the art

assets that we'll be making use of today. that we'll be trying to build today assets that we'll be making use of is this one called Oakwoods Environment Asset.

And this is by brulav. And this is available on each.io and it's completely free to use.

so what you wanna do is just click on download and over here you can choose to donate a certain amount to support the developer, or you can just hit on over

to this No thanks to the downloads link. Once you're over here, you click on download and this

should bring up your file explorer.

I'll suggest putting it into the assets folder that we just created and that you have it over there.

Alright, So you can see here I've put this into the public assets folder, that we created earlier.

And all you have to do is just extract this right now.

So I'm gonna delete the zip folder and take a look what's inside. I'm also gonna change this just to say oakwoods because it doesn't

need all the, additional, prefix. So you're pretty much done in terms of getting hold of the assets.

switch back over into Via code and you will see that the assets are sitting over

here, and it's worthwhile taking a look at the type of assets that we're working with because this is really important when you're working in game development.

So if you go into this character folder, you'll see Char Blue, png. And if you actually see what this is something called a spritesheet.

So what happens is that animation is played by playing one frame two frame three, frame four frame. Five frame six frames, and you will see this one is harder to look at, but

the first one is the idle animation. The second row is the slash animation, and the third one is the running

animation and so on and so forth. So you will need phaser to understand how to read this spritesheet in

order to play the animations. in the past, without ai, you would have to program this manually in order for it to understand that the first row is idle, the second row is slash, the third

row is running, and so on and so forth. But that I, we'll be showing you right away. You don't need to do any of that. Thanks to the power of ai.

Now this is all done automatically for you. Okay, so let's take a look at something else. So you can see this background folder over here.

This one shows you the three different backgrounds. That are available. And the way that this works is that there are three layers to the background

so that you can get the nice parallax effect that you see over here. So let me just jump back into the game and you will see that as I run,

you will notice that the background actually scrolls at a different speed. this is called parallax scrolling.

And the way that you achieve that is by having three different layers of the background when you're running, it moves them at different speed.

So it gives you a sense of Okay, so this is one of the other tricks that game developers like to use. Alright, so jumping back into Visual Studio code over here,

the next one we wanna look at is decorations you will see that you will have placed within the level. These are the simplest because all they have to do is just

load it in and place it there. There's no animation and it can just be loaded in like a static object.

And then the final one is something called a tile set. Now if you look at this is, how you would piece together the entire level.

So there are different parts. Think of it as like little Lego blocks. And In order for you to construct something that looks like

this, that you see over here, Which piece of this talent to be loading at any one time and in order to do so

in the past, like I said, you would have to create some kind of level editor. But as I'll be showing you, you wouldn't need to do that with what we are doing today, Okay? So that's the introduction to all the assets. Now gonna open up cloud code, as you see over here.

And we'll be making use of just two key prompts in today's tutorial. so what I've done is that I've also provided the prompts that we'll be using, in this folder called prompts over here. And let's take a look at the first one. The first one really just says that it's going to be a way for us to

Step 3: Create assets index

build something called an index, JSON. And this will allow the AI to know what assets are available to it

in order for it to build its game. And all we need to do is just copy this prompt that you see over here and paste it into cloud, alright?

And you will see all it's saying is that study the assets in the Oakwood folder and then create an assets JSON that is an index to the assets

account for the spritesheet animation. account for the tile sets and then so that we can reference these assets for phaser.

And then what I'm gonna do is say use the phaser js skill. and before we go ahead and kick this off, type slash model and make sure you're using Opus 4.5. In fact, just as I hit this off just now, I was actually using Sonet 4.5,

which is not what you want to do. So Opus 4.5 It's the best model and is exactly what you wanna do for a task like this.

Okay, so we are just gonna run this command one more time, and while this is running, I'm going to actually, make use of a tool that

I've created in order to help avoid mistakes like that in the future. So I'm just gonna open up a new terminal over here.

And What I'm gonna do is just type NPX at chong UC cc status line

at latest in it, and you will see very shortly what this will do. this effectively will add something called a status line to your clock code

so that it will show you what model you're using and several other things. That will help you to avoid, the bug, mistake that I made earlier.

Okay, so this has been installed and this will only pick up once I've restarted Claude. So we will wait until this stat completes here.

But you can see that what it's doing is that it's looking through the folder that we had over here with all of these assets, and then it's

gonna create a single file index. in order for it to know what assets are available. So once this has been created, we'll draw back and then we'll

talk about what's happened there. Okay, so you can see that we are now done with the, assets JS creation.

So very quickly, I just wanna show you what this has achieved. If you take a close, look at what is being stated over here.

It says that it has summarized all of the different assets. It knows that there are three

backgrounds.

It knows that there's a character. It knows that there are decorations. even the shop is animated and it knows about the tile set.

And I think it's really important to understand why this is really important for it too. Get right.

So let's take a look, right? It's been able to automatically find out that there's idle run, attack one,

attack two, jump, fall, roll, and death. Okay? And so if we actually pull this up right now, you will see that

this is indeed correct, right? it's very important to double check this because sometimes you will notice

that it may not always get it right. You can see that it says that the idle animation is from zero to three and then it says the run animation is from seven to 13. And you will notice that This doesn't look exactly correct and I've seen

this sometimes get it right the first time in other cases it actually will make small little mistakes like this.

Okay, so let's actually take a look at this. All right, so the way that you will see it is that this is 0 1, 2, 3, 4, 5, 6, 7.

All right? And then after it goes 8, 9, 10, 11, 12, 13, 14, 15, okay, so I will say that this is probably not going to be right, And so I'm gonna challenge this and just get this right and I'll probably update the skill to be more aware of this.

And remember earlier I said that, you would have to do this manually, but in this case it's managed to automatically identify different animations and the frames, but sometimes they can still get things wrong. So let's try this very quickly.

I've taken a look at the assets, Jason, and what you've said, and there are a few mistakes. All right. Looking at the character, the idle animation actually goes from frames zero to five, and then there are two frames empty in frame six and seven.

The attack runs from 8, 9, 10, 11, 12, 13, so it goes from eight to 13, and then there are two empty frames in nine and 10. verifying that this is the correct understanding that I have, and then make the, appropriate fixes.

Okay. so I think this is quite important to get right, so I'm just going to tell it. having done this so many times now, I always find that it's quite important to get at least some of these fixed correctly. because otherwise, in some cases, the errors that you see may not be as obvious.

Okay, so it's going to study the spritesheet a little bit more and then verify whether or not our understanding is correct.

overall you can see, that it's quite good. So let's just take a look at the tile set. It says that it's a 21 by 15 tile so we go 0, 1, 2, 3, 4, 5, 6,

7, 8, 9, 10, 11, 12, 13, 14, 15. 16, 17, 18, 19, 20, 21. So it is correct.

It is 21 by 15. So it's got that correct and it knows about all the different tiles. Okay, let's take a look at what it says

and. Yeah, so I think it actually got, so I, all right, so he's gonna do that.

So 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. Yeah, so actually it did identify that I made a mistake with,

the empty frames in nine 10. I think it's because, I'm using this per flow, so as I was counting, it actually got the numbers wrong. So it seems that yes, actually understood, the mistake that has been made and it has now got it. Correct. Okay, perfect. So now that we've made, this fixes, let's go ahead and implement the game.

Step 4: Plan Mode

So now what we are gonna do is that we're gonna switch into plan mode. I'm gonna hit on over back into this art asset and I'm gonna grab this and I find that this is actually quite useful to do. So if you, save this image here and I'm gonna call it mock up.

Okay? this will be very useful for the next step, and the reason for that is that sometimes even though he has all these individual assets.

Without a mockup, you will not be able to get a good understanding of the scene And I've shown this with three Gs as well.

in certain cases, you do not have mockups, you just have the sets and the different character elements. the trick here is that you can just go into Nano Banana Pro, give it the tile sets, and then ask it to generate a mockup for you. And this is exactly what I did in my last video, but in this case, we have a good mockup.

So let's just make use of this. Great. So with this being done, we're gonna use the second pro over here.

Okay. So I'm gonna say let's use the phaser G skill to replicate the image. And I'm going to change this so that you actually reference the mockup.

Alright, so you've got a mockup here. Plan it out first. add the three backgrounds, layers first, then. The tile set for a flat ground, in a single layer, and then place the character and animate it with controls. And then that's the baseline. So I think what I'm gonna do is step one, step two, step

three, and that's the baseline. And then remember to add this one where we say to reference the assets

to Jason for the list of assets. and you can see up here, I'm saying use the phaser J skill so that it is going to be able to, verify that this is working right? Okay. now we're gonna use plan mode and I activated using shift tap because

I want this to occur step by step. And this is the advice that I'll give you rather than trying to one shot everything.

I think it's really important to always try to think of game development or any software development, for that matter as a step by step process.

One short thing is fun, just as a demonstration of the capabilities of the model, but in actual development, you want to take things step by step, and this

is the best advice that I can give you. and the reason for that is that inherently, development is always iterative and it's a lot easier to fix box as you see them along, as opposed to it trying to, build so many things at the same time and then end up with lots of different things that.

Affecting one another. Okay. Alright, so We're gonna let this plan, run along and then once it comes up with a plan, we will, draw back and see what we're gonna do. so what you are actually seeing here, if you take a look.

Step 5: Testing with Playwright

Is the background correctly laid out with the various, layers correctly, And I'm just gonna jump in here to show you what's going on, but you can see that, in addition to implementing the background, it is making use of the playwright MCP server that we saw earlier to open up the URL and then taking a screenshot so that it can verify whether or not the.

screen looks correct. this is exactly why having the playwright MCP testing server is going to be very useful, especially when you're building something like games.

'cause a lot of times in games things are going to be visual and as you can see here, it's already created a character as well.

And the character's just jumping on. And what it's doing is that, I'm just gonna jump back into Visual Studio over here and you can see added the character with physics, and then it's gonna take a screenshot. And you can see, take a look over here. It says, phase six, verified character is visible and standing on the ground

with physics collision working. And indeed that is true, right? Because you can see that their character is landing from the

sky And then it's landing on the ground that you see over here. So this is the process of why using agent skills as well as the current

MCP service can be very useful for things like game development. And now that it's created, that it's gonna go ahead and, add the

animations and the input controls. Alright, so now it says it's done it. And this is the really interesting part because what you will see now is

that it's going to check to see if the animation is playing correctly, and then it's gonna see if the controls are working correctly as well.

So what playwright is gonna be able to do is that it's going to be able to press keys, as you see over here.

Okay? And then, I think the best way to show this is that if I actually make this screen a little bit smaller, okay.

So that you can see. Because what you will notice is that playwright is going to actually play my game for me, in order to determine if the controls are working

correctly and in order to determine if, things are proceeding as planned. I'm just gonna drag this over here very quickly and just take a look.

I see that there might be a rendering artifact. Let me reload and test the jump. Okay? And then it can say here, let me test the movement.

Controls pressing right arrow to move the character. Okay? So the first thing you notice is that there's a rendering

artifact and indeed, right? Okay. So there's a couple of issues, that we want to fix over here. So it's tried to use playwright to, perform a bunch of different

tasks, but it's still managed to get some of these things wrong. And this is the interesting thing, that I've seen is that sometimes

it's really smart and it's able to see hey, wait a minute, there's an artifact and I need to fix it. And other times. It tries to think very cleverly, and I've seen this more with

Opus 4.5 than with, GPT 5.2. We actually managed to get, to the state, that we've, all seen

before.

So let me So let's just pull up this, one that we see over here. So with GPT 5.2 I, it was actually able to get hold of the either animations with no issues. so it's quite interesting to see. And remember, I actually did say to go step by step and oh, 4.5 just went ahead and, did not let me verify, each way. it was able to get things to a pretty decent state, right? Because, you can see movements is working. the, the background, the background is layered correctly as well. But, just a bunch of things that would have been better for it to verify before, we move ahead. So the first thing we're gonna fix is this, idle animation. So it's definitely got the framing wrong, so I'm gonna.

Step 6: Bug-fixing

just see if we can figure out what's going on over here. There is an issue with the I animation. It looks like the frame size is miscalculated.

remember to refer to assets J and, fix the, visual, issue.

Okay? So I think that's the biggest issue that we want to fix over here because everything else is actually quite okay, like it's managed to layer the background correctly.

and it's managed to get the controls in. It's just the rendering of the player is not correct. And even though, playwright managed to do quite a good job

of verifying different things. It did happen to get this one wrong. Okay? So it's not perfect. And like I mentioned, I've had more success in getting things,

working correctly using GPT 5.2, with the previous version. So this is, something to, observe, open 4.5 is indeed very good and can

get, very far, but oftentimes, it can, try to be too clever and move ahead,

a little bit too quickly Okay, it's managed to fix it, appropriately. So if you look over here, I'm just gonna make this, screen a little

bit bigger so you can see, it is indeed working correctly now. All right, so the i animation is playing correctly now and the movement of the

character is, working as expected. So there's no longer this strange, double counting. And I'm gonna show you something really interesting about why it

was not able to spot this error. Earlier. So what happens, first of all, let's just jump in here, all right?

And just take a look. So I'm gonna show you what it says. All right? so as part of its testing, it says, I can see the issue.

All right? The ghost artifact, appears to the left of the main character indicating that the frame width is incorrect.

And you can actually take a look at what this is seeing over here. So this is the, this is, you know what Claude has assessed.

So he says, Hey, wait a minute, like this thing that we see over here, is just not right, and you can see that playwright, continues to save all of

these screenshots in order to verify things that are working correctly. And the reason why it thought it was working correctly is that

look at this movement, right? screenshot. it thought that the movement, to the right was working

because, that the character has moved to the right, but actually it was because of a bug earlier. so this is the reason why, it's not a foolproof, system. it can introduce, errors, but, hopefully it shows you. how clever it can actually be if it doesn't have a cascade of, different bug there that you see. Okay, so this is working correctly now. it was just simply a calculation that the frame width that was wrong. And I have updated the skills, since recording, to try to avoid this error. But, sometimes it still gets it wrong. if you actually look over here, as part of the skills it actually, tries to avoid, miscalculations, All right. but in this case it'd be quite a small little fix, but we are in pretty good shape now. Okay. So I think, overall it's actually looking pretty good.

Step 7: Iterate

make the last few changes to this to get this working correctly. Okay. So what you'll see is that the, animation is working correctly, but, the run animation and the giant animation not playing right? And the last thing we wanna do is that we want this, map to not just be a single. level. We want this to be infinitely scrolling to the right with parallax. So let's just do that one first.

The background currently, looks great, but I would like, parallax scrolling and infinite, ability to move to the right.

Okay. I'm gonna use the phaser, skill, to get this done. Okay. So let's take a look and see if this will be able to get this

to, work infinitely as well. All right, because, if you recall, in this version here, we actually have, infinite scrolling, with Parallax as well, which is a really nice effect. And, it's just a little thing that, ends up making the game

feel a lot, more polished. Okay, so let's, jump back to, alright, so this is what we are hoping to be able to do, and we'll see, if it's gonna be able to get there. Okay. All right, so let's take a look. Alright, so I'm gonna just expand this really big now so you can see it is, infinitely scrolling to the right now.

the, there is a parallax effect happening with, the trees, the because it's repeated with just a single tile, it doesn't look very smooth,

but we can fix that up, quite simply. but overall you can see that this is moving in the right direction.

Alright, Okay. Actually it's bad practice because I'm, if you see this here, this gray version of Chrome is actually, launched by, clock code in order to figure out if things are working correctly. it says that scrolling is working All right. No right boundary, you can keep moving, right? So that's correct.

So let's fix up the animations. Now, the play animations are not correct. Idle is perfectly fine, but moving is not, and jumping is not fix them.

Okay. And we are reaching dangerously close. with, contact, in fact, with 45% contact is probably a good

time to just compact it up. But you know what, let's just push ahead and I think in the next round

we'll definitely want to compact our context.

Okay? So let's just give this a shot and see if it's gonna be able to fix this. Okay. And you can see here that the, playwright is, actually testing, the run animation automatically. and it's taking screenshots as it goes along. So because animation takes, a while to render, so it's taking multiple screenshots and immediately you can see that, it's identifying that there is a bug with the animation over here.

So they say that the duplicate was likely a rendering artifact. So this is really interesting. So what I'm gonna do now is, because there's only 37 context, I'm gonna do compact it. the compact is completed. Let's see if it's gonna be the fix this issue over here. So I'm gonna say the, run animation is, still wrong.

it seems that there's a bleeding, from the frame. so if you take a look at the screenshot, it's loading the

top part of the frame above. and it's causing the height to be wrong.

Okay. And then we just draw in the, just put this image in and see if it's gonna be able to fix this.

Okay. because we compact this, you'll see context remaining is 37% for now, but it should update to show you the latest context, quite shortly.

Okay? All right, so you saw that, all right, because the context, needed to refresh once, clock came back. So we actually managed to reclaim, almost, double the context.

So 78% is a good context to have, so that it doesn't end up having context fraud. Okay. So you can say it's looking at the screenshot, that's bleeding

from the row above into the run. This suggests that the frame height, calculation might be rough.

Okay. like I said, this is, quite odd because, I have done this with Opus previously

and it did not, have this issue. So this is, I guess just an unfortunate run. it tends to get this right most of the time.

but, let's see if he's gonna be able to fix it now. All right, so you can see that, this is working correctly now. Okay, so the jumping is working as expected and the running is

Step 8: Update skill as you go along

also running correctly, as well. the reason why this was wrong is that, there was an issue with the frame

and the frame height for some reason. in this run, it got the frame height wrong, and the moment you get this

frame height wrong, that's where you get this very strange artifact, going on. So this is a quite interesting thing because, in different runs,

I've never had this problem. And, it just so happens that, just looking at this and looking at a previous, json that I had, this was the issue.

So this is a really important point here, right? Whenever you're using skills, and I've been updating these skills along, sometimes you can introduce regressions.

So I'm going to do something here. I'm gonna say. how did you end up, getting that wrong?

It never happened before. Please, look at the phaser game Dev.

Skill and update it so that this mistake doesn't happen again.

Alright, so this is the step that you wanna do as you, skills are not a onetime work kind of thing.

You always want to continue to improve it as you continue to build more projects. And like I said,

the skill that I've been using, in this one is actually a skill that I've. It on since I last recorded the video, so I might have, caused a bug in it.

but this is the way that, it works. So while this is going on, let's actually continue looking over here. Okay. So you can see, that the game is working fine.

I wish this was full screen, but it doesn't really matter. That's actually just a viewpoint issue. But you can see the background is working, the character is, also animating correctly. And then, the only minor things that you wanna say is that a little bit lower as

we see over here, and the player should really be sitting on top of the tile. So these are minor things, but ultimately, it's got into a pretty good state.

and I think like the main issue, as we saw over here was just really about animations.

Everything about the controls, and how the animation states.

was running, and linked with the controls was all correct, was just, it got the frame wrong. So it's just one tiny mistake can result, in the, it looking, pretty bad.

Okay, so let's just see what it says over here. 'cause I'm actually interested. Alright, so it says here that a quick little note about square things being common and then added check for square frames first, non square dimensions for character Sprite and assume roll count matches, column count.

So this is actually one, of the reasons why he actually got the calculation wrong. Okay? he actually thought that this was a square.

And, in this case, let's actually just take a look. but if you actually check the dimensions and you see over here, right?

the dimensions is 4, 4, 8 by 3 9 2. So you got it wrong. And, this is just one of those mistakes that was introduced.

Okay? But, otherwise it actually got pretty far. So let's actually finish this up. Okay? Just a few more things to fix.

the ground tile is a bit too high. she needs to be lower as you can see, and take a screenshot once that is fixed to verify. Secondly, the player should be standing on top of the tile.

right now he's sinking into it a little bit. Alright, let's fix those two things and hopefully we'll get to a good state.

and I am assuming it's gonna get this right, this time around. but you never know. You can be surprised sometimes how he can get, some of these things wrong.

Alright, so it's tried to adjust the ground position and, you see

that, I'm not sure whether you saw it, but it is correctly putting the player on top of the ground now.

it's still a little bit low, but it's gonna take a screenshot. So he says, the ground tower is lower. And it's still not quite right.

The, ground is still not anchored to the bottom of the view and I'm okay with the player sinking into the ground a little bit.

I think that's totally fine. So let's not get too nitpicky, about that one. And you can see that it's actually moved it high out now, which is really odd.

Step 9: Automated test-implement loop

and I'm gonna actually steer it midway. You can actually do this if Claude. it's trying to be too clever now. I'm like, I'm gonna stop and say stop what you're doing.

Keep a single roll. Just simply move the ground tiles, lower.

So you can see here it's trying to, move, the towel lower, but it actually ended up raising it a bit higher.

So it's gonna take a screenshot and then you can see it's right over here. So hopefully you'll be able to figure out that, Hey, wait a minute.

you actually made it higher rather than lower. So you should be able to fix this. Alright, so you see, he actually did spot it. It says, there is a gap at the bottom, so it's gonna try to adjust, offset more.

then it's gonna continue to, offset it until it gets to the bottom. So this is, like the process in which it will take, in order

to get this working correctly. Okay. so it would just continue to be able to, load the game and then take a screenshot and then adjust from there.

So this is like automated testing to some extent. playwright is quite useful for something like this, but, it can be a little bit slow.

sometimes you've gotta be a bit patient, especially when you're dealing with things like around the map. Okay. All right. And it, you can see it says with this value, it's much closer to the bottom.

So it's gonna try to push it even further down to the very bottom edge. I'm gonna show you over here.

As it continues to adjust the values, it's automatically taking screenshots and it's lowering it. So to the point that it's actually low enough, to get to where we want.

finally, I'm gonna say, final thing. Let's add an action button where the character plays the slash animation.

And I'm just gonna say, use the, fasa. Skill and remember the s Jason, it's because we are running low on context

and it's just really useful just to make sure that it, it's aware of, the things that you learned before. it's gonna add the, attack animation with space bar and then, it's

gonna track the animation state. So let's just see, how this works. Yeah. We go, yeah, I'm pressing space button now and you can see

that this is working correctly. Yep. Alright, so it's got into where it needs to get to and, And, it's just

gonna do a bunch of, a little fixes. Alright, So the attack is working now and, we've pretty much gotten to

where we need to Obviously there are little tweaks that you want to make. for example, the, distance, between the foot and the ground.

and there's also this little gap over here. But these are small little things That you can always put later. thing that we're gonna do is that, alright, so now let's add, some

Step 10: Polish

of the props and decorations into the level to make it look better. Alright, so this one should be a pretty straightforward thing where

it's just going to, just, add some of the elements, within the screen so that, it actually has, a

visual.

So it looks, actually, it looks a little bit less empty. 'cause right now all we see is just the ground, and then the trees in the background. So it'd be nice to have a bunch of props.

So you can see here it's loading up and this is what I told you, like I told you earlier, right? It's gonna look at the assets that Jason, and then it knows all the decorations.

So this is just a much faster way. It didn't need to, go through the file system in order to then try to figure out, ignores everything that's available, straight away. alright. Can see here it's, put things down.

but everything's floating. alright, so you can see here, it's gonna take a screenshot, alright? in order to verify if everything is looking correct.

and you can see straight away it says that the decorations are visible but are floating above the ground.

So there you go. It's gonna start moving things. so he is gonna continue to try to move the props down so that, they

are not, floating above the ground. Okay. And it's gone ahead and, it's, managed to move everything down.

I did have to guide it a little bit, because it was, still not quite getting under the ground, which is quite interesting and, but

the level looks much better now. Okay. As you can see. Cool. Alright. so there you have it, right? you can jump.

there are props all around. So it's quite close to, the state that we had earlier. Like I said, it did struggle the most, as we saw with two things,

which is the, it got the frame size wrong, the moment that gets wrong, the animations all walked, and, quite buggy.

but then, in terms of like getting things together, once that was fixed, it was all just little, nitty gritty things like, like the distance to the

ground and, small things like that. and yeah, this brings us to the end of the tutorial. in my opinion, actually doing 3D games is a little bit easier because when it

Wrap Up

comes to 2D art, you will notice that, it's really about how the assets are set up and there's no real standard way, for setting up these assets.

you need to make sure that you get the dimensions right and AI can get pretty far automatically and help you with a lot of the legwork.

But if it does make a little error here and there, it can compound quite quickly. but overall, hopefully you found this tutorial useful.

I'll be continuing to iterate on these skills and continuing to try to build more 2D games as well as 3D games going forward.

so if you enjoy today's tutorial. Give this video a like, hit that subscribe button and don't forget to turn on notifications so that you always be the first to know

whenever new content like this drops. And oh, if you haven't seen my previous, tutorial, where I actually do a very similar process, but for 3D games using three Gs, you might wanna check out this video right over here. Alright, till next time, I'll see you.